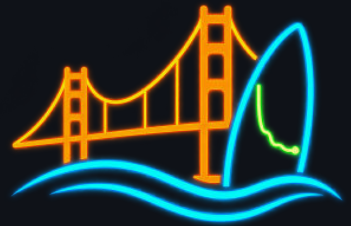
LMCACHE WORKSHOP / OCTOBER 22, 2026



Improving inference through byte movement

Is AI model prompt caching important today?
Asymmetric KV quantization
LMCache DMA-BUF / io_uring
Offloading beyond KV Cache

Luis Chamberlain (@mcgrof) Samsung Semiconductor
Senior Principal Engineer

How can Samsung make a positive impact for AI inference?

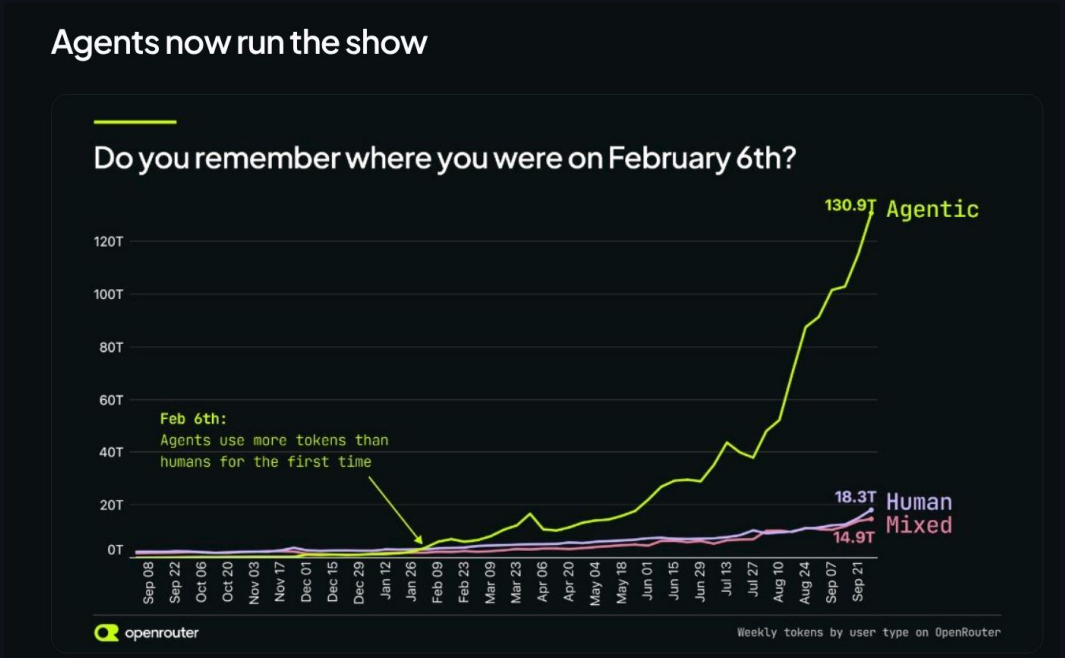
$$\text{Arithmetic Intensity} = \frac{\text{FLOPS}}{\text{Bytes move to/from memory}}$$

We should work on industry point points which Samsung can control

Is byte movement a real pain point?

Is caching a real pain point?

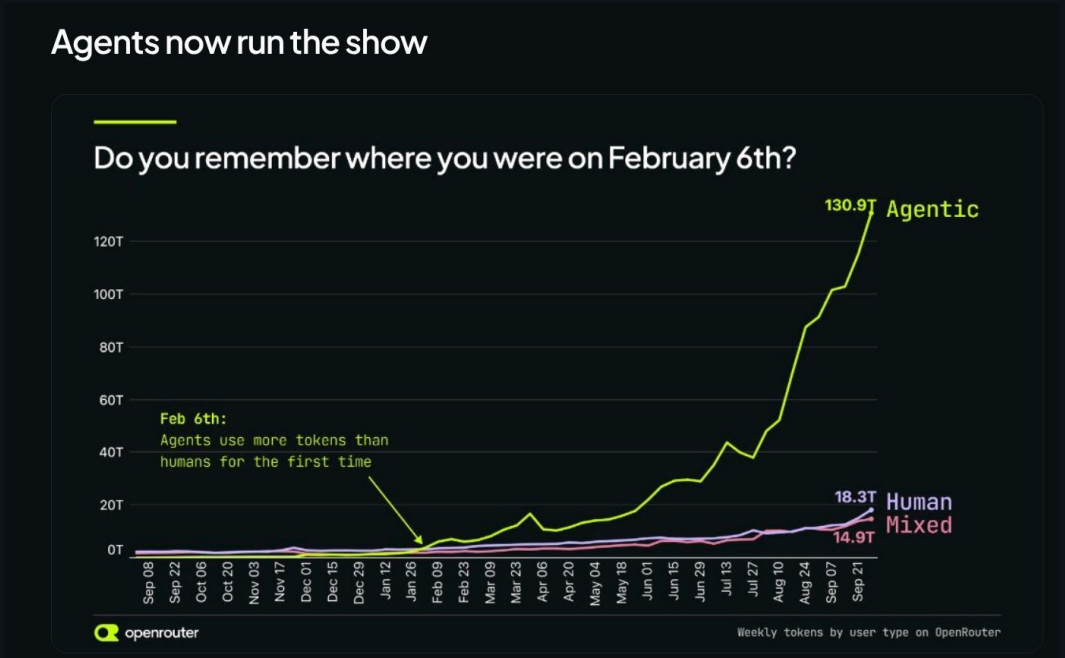
Trend: agentic workflows lead token use



Weekly tokens by user type on OpenRouter.

On February 6th, agents used more tokens on OpenRouter than humans for the first time. In the last week of September, agentic API keys used about 7x the tokens from human API keys. This gap is likely to continue to widen over the coming months.

Trend: agentic workflows lead token use



Weekly tokens by user type on OpenRouter.

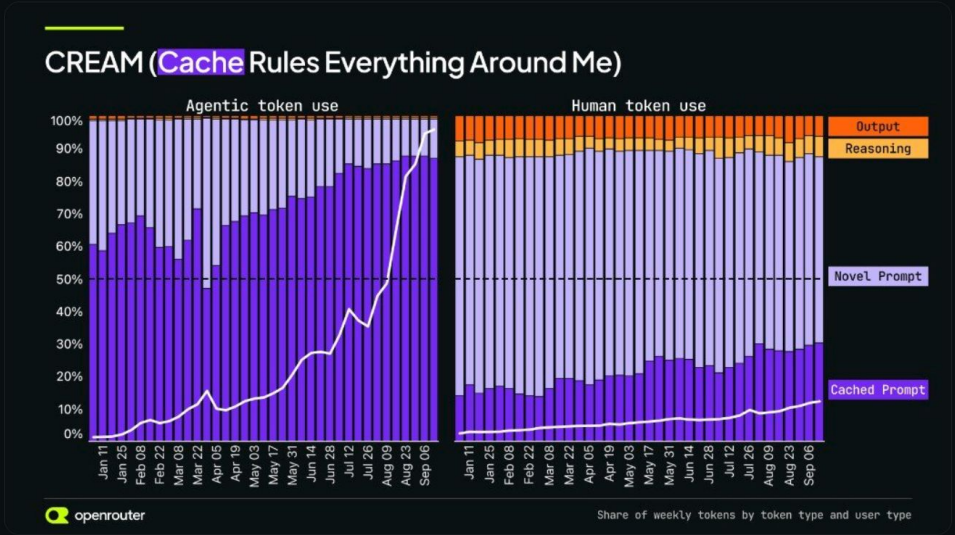
On February 6th, agents used more tokens on OpenRouter than humans for the first time. In the last week of September, agentic API keys used about 7x the tokens from human API keys. This gap is likely to continue to widen over the coming months.

Who cares, how does this relate to byte movement and caching?

OpenRouter October analysis

Agents ❤️ caching

Cache rules everything around me



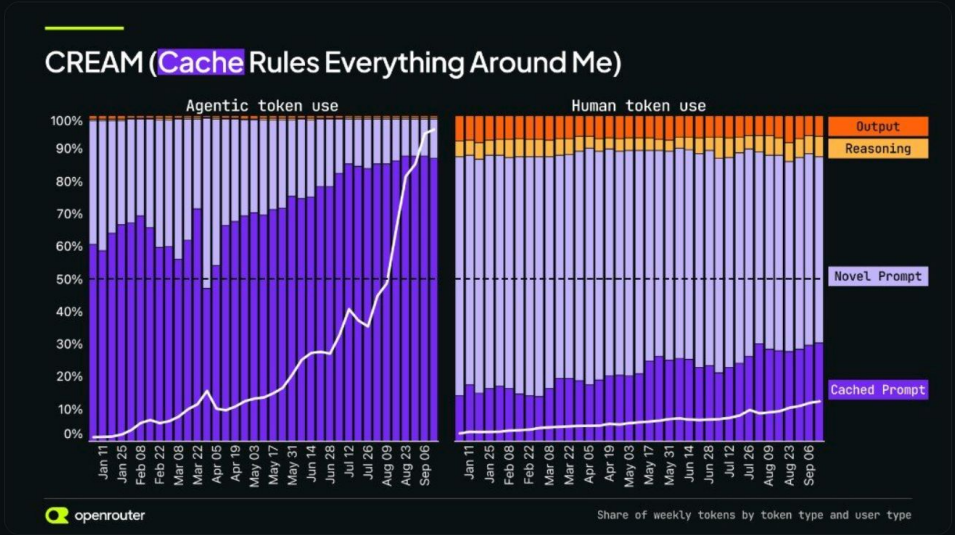
Share of weekly tokens by token type and user type.

Agentic workloads look nothing like human ones. An agent re-sends its whole context on every step: the system prompt, the tool definitions, the files it has read and the history of every tool call so far. Each step adds a little new information and asks for a short reply. So the overwhelming majority of agentic tokens are prompt tokens the model has seen before.

OpenRouter October analysis

Agents ❤️ caching

Cache rules everything around me



Share of weekly tokens by token type and user type.

Agentic workloads look nothing like human ones. An agent re-sends its whole context on every step: the system prompt, the tool definitions, the files it has read and the history of every tool call so far. Each step adds a little new information and asks for a short reply. So the overwhelming majority of agentic tokens are prompt tokens the model has seen before.

[DualPath: Breaking the Storage Bandwidth Bottleneck in Agentic LLM Inference]

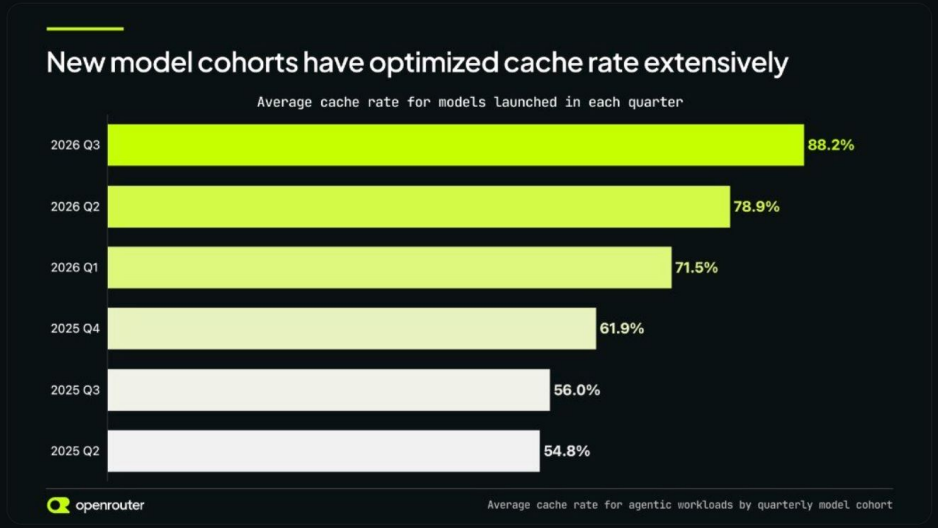
February 2026

95% Cache hit rate

OpenRouter October analysis

New models and providers are optimizing **cache rates**

The industry has optimized for cache

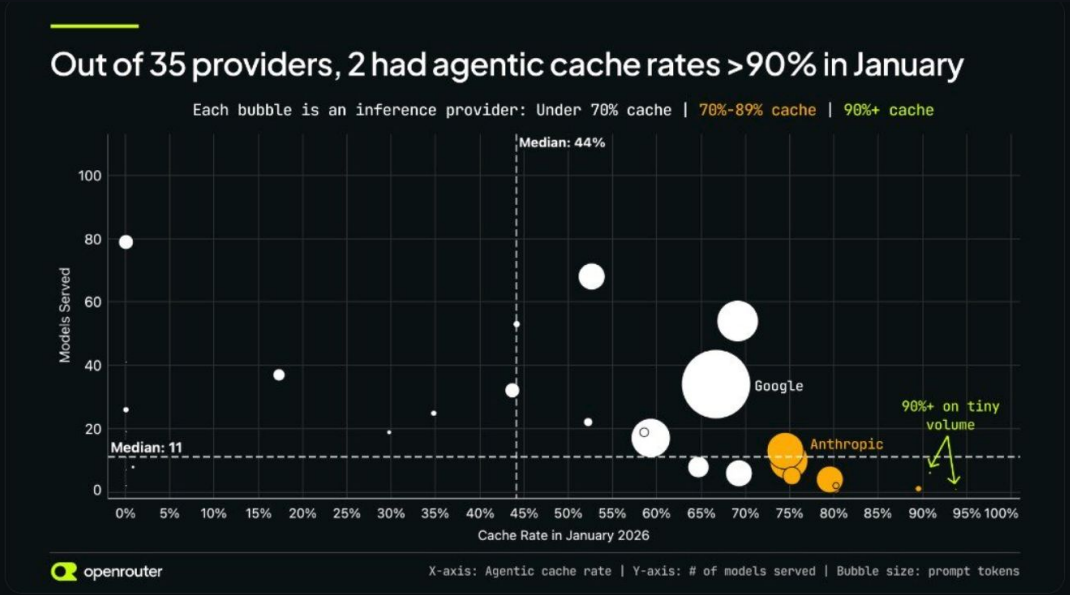


Average cache rate for agentic workloads by quarterly model cohort.

The economics of AI have pushed the entire industry towards improving cache rates. Models launched in Q3 2026 averaged an 88.2% cache rate on agentic workloads, up from 54.8% for models launched in Q2 2025.

All layers of the stack, from the OpenRouter algorithm to the model makers to the inference providers, should receive partial credit for such impressive acceleration.

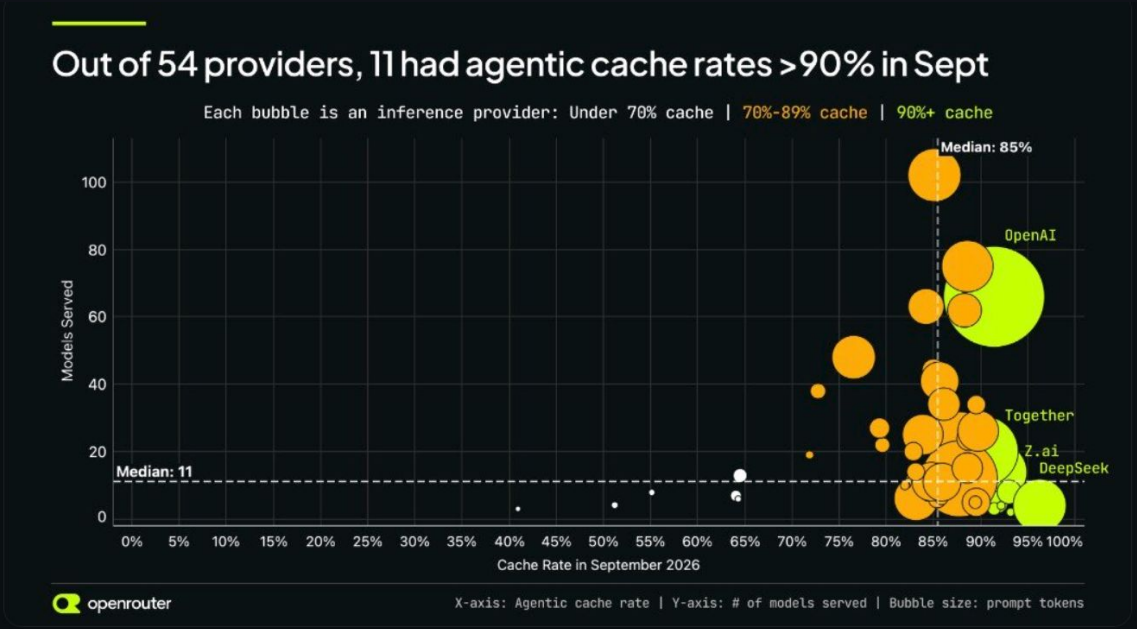
cache rates ~ January 2026



Agentic cache rate by inference provider, January 2026. Bubble size is prompt tokens.

The inference provider picture has undergone particularly rapid change. In January, only 2 of 35 inference providers served agentic traffic at a cache rate above 90%, and both did it on tiny volume. The median provider cached 44% of agentic prompt tokens. Even the biggest, Google and Anthropic, sat between 65% and 80%.

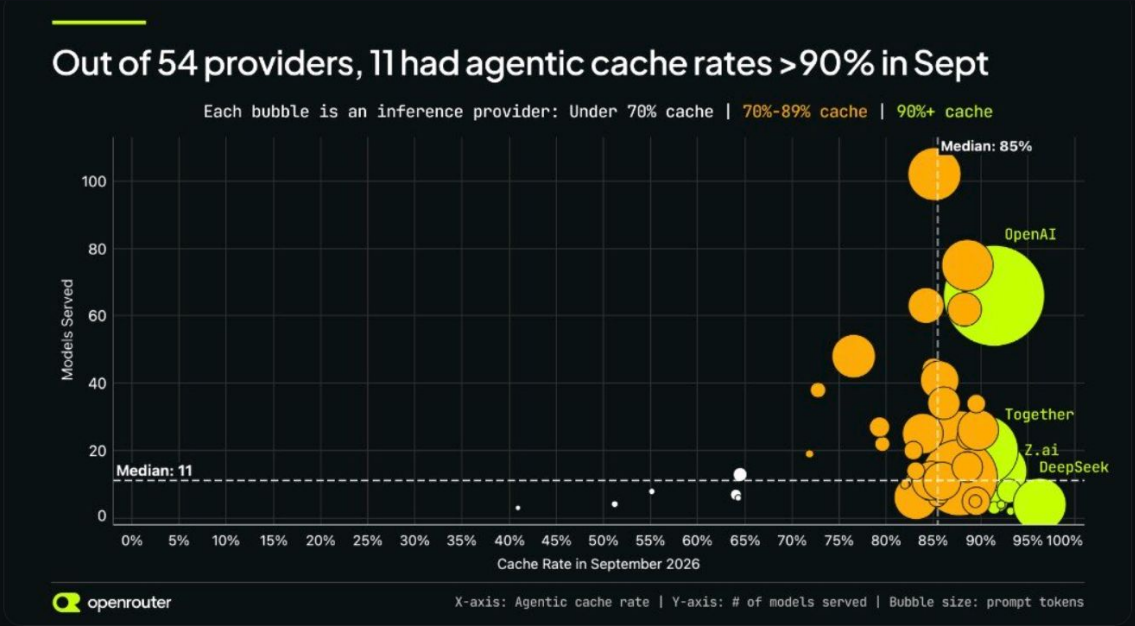
cache rates ~ September 2026



Agentic cache rate by inference provider, September 2026. Bubble size is prompt tokens.

By September, the median provider was at 85%. 11 of 54 providers cleared 90%, including OpenAI (the largest bubble on the chart), Together, Z.ai and DeepSeek, which led the field at around 96%. Only a handful of small providers are still below 70%.

cache rates ~ September 2026



Agentic cache rate by inference provider, September 2026. Bubble size is prompt tokens.

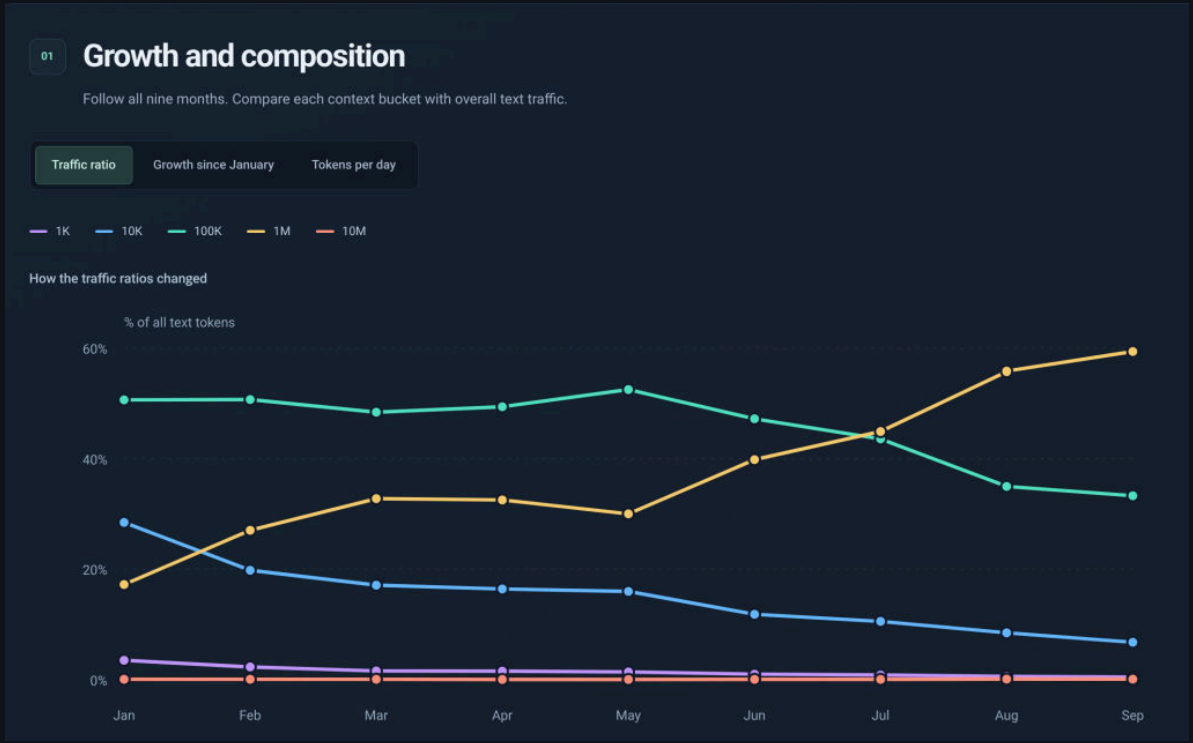
By September, the median provider was at 85%. 11 of 54 providers cleared 90%, including OpenAI (the largest bubble on the chart), Together, Z.ai and DeepSeek, which led the field at around 96%. Only a handful of small providers are still below 70%.

How about input + output token size?

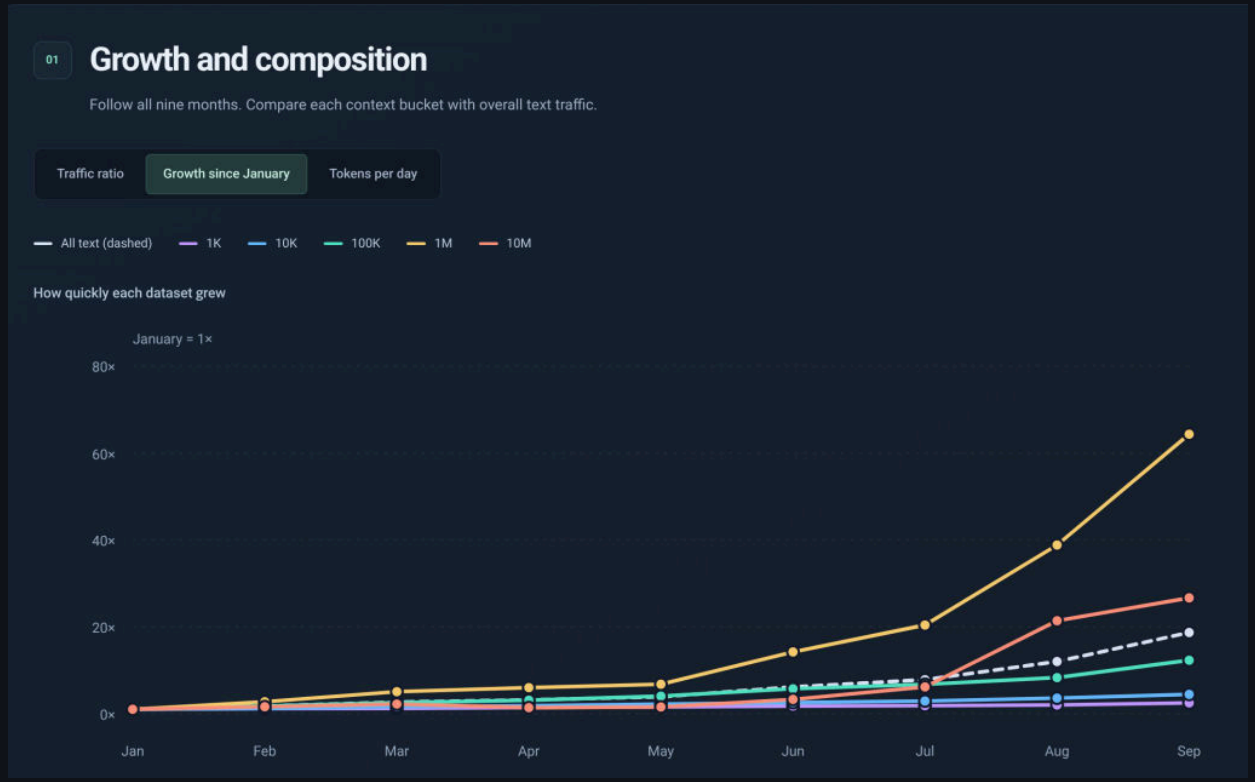
How about input + output tokens per API call?



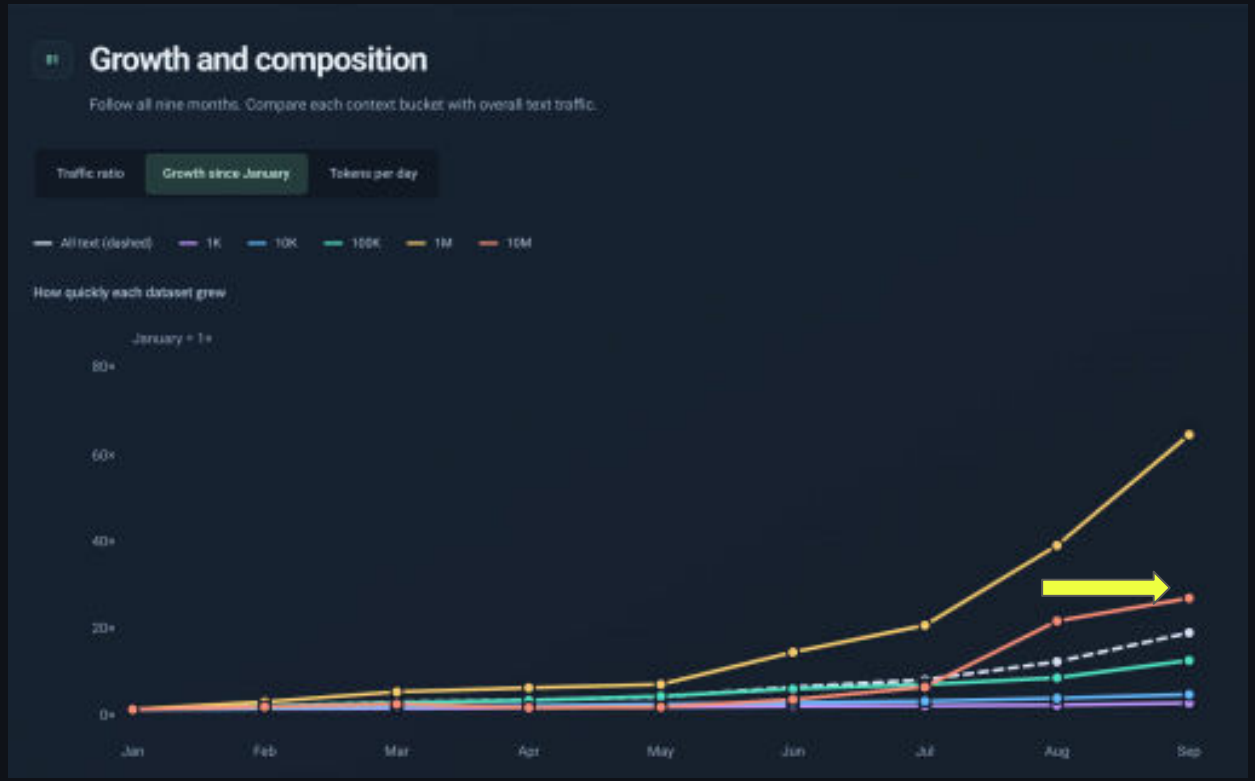
Token ratio by context length



Token growth rate by context length



Token growth rate by context length



Wait…

Is that 10M million moving up so fast too?

OpenRouter October analysis

Token growth rate by context length



LMCache project:
kvcache.io

Wait…

Is that 10M million moving up so fast too?

Quantize keys and values for stored activations

Reduces KV cache sizes

The good:

- Reduces disk storage
- When fused reduces GPU memory consumption

The bad:

- Quality can be impacted
- It fails on *some models*

Symmetric FP8 for both K and V can fail

31×
PPL increase at 2K

202×
PPL increase at 8K

0%
Retrieval at 32K

Qwen-2.5

FP8 Symmetric Quantization failure root cause analysis: Qwen2.5-7B / H100 - **Model and format both matter.**

Root cause: models with bias in attention

Pre-bias repairs quality

Fix idea: unified diff (simplified)

```
--- naive_fp8.py
+++ prebias_fp8.py
@@ -1,3 +1,4 @@
 k = k_proj(x)
-k = rope(k)
-k = fake_quant_fp8(k)
+b = k_proj.bias.to(k.dtype)
+k = fake_quant_fp8(k - b) + b
+k = rope(k)
```

Subtract bias, quantize residual, restore bias.

Retrieval repaired

0.00 → 1.00

Needle retrieval at 8K, 16K and 30,720 tokens.

Optimized serving kernel

0.654× requests / second

2.056× p95 request latency

Qwen2.5-7B / H100 - Attention bias aware fix attempt

WHAT ABOUT K16 / V8

Quality, Payload, capacity and speed are different

Empirical quality results: K16/V16 ~ K16/V8 GSM8K

4 bytes
K16/V16

3 bytes
K16/V8

2 bytes
K8/V8

Payload. K16/V8 saves 25% for equal-sized K and V.

Capacity. 1/0.75 = 1.33

- About 33% more concurrent sequences at the same cached length
- About 33% more cached tokens per sequence at the same concurrency.

Speed. Needs measurement.

THE QUESTION / WEIGHTS AND KV ARE SEPARATE

Does asymmetric KV quantization still help models with INT4 weights?

INT4 describes the weights, not the KV cache.

AWQ = Activation-aware Weight Quantization → not all weights are equally important.

- ***There is a small fraction (0.1%-1%) of salient weights; skipping the quantization of these salient weights will significantly reduce the quantization loss***
- Uses sample activations to calibrate 4-bit weights before deployment.
- AWQ commonly means W4A16: 4-bit weights and 16-bit activations.
- The original paper uses FP16 and the authors' implementation subsequently added BF16 support.

Test: Qwen3-8B-AWQ. Keep the same INT4 weights and change only the KV recipe.

THE QUESTION / WEIGHTS AND KV ARE SEPARATE

Does asymmetric KV quantization still help models with INT4 weights?

INT4 describes the weights, not the KV cache.

AWQ = Activation-aware Weight Quantization → not all weights are equally important.

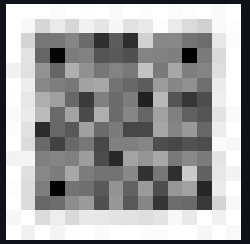
- ***There is a small fraction (0.1%-1%) of salient weights; skipping the quantization of these salient weights will significantly reduce the quantization loss***
- Uses sample activations to calibrate 4-bit weights before deployment.
- AWQ commonly means W4A16: 4-bit weights and 16-bit activations.
- The original paper uses FP16 and the authors' implementation subsequently added BF16 support.

How about Asymmetric quantization with K16 / V8 ?

Test: Qwen3-8B-AWQ. Keep the same INT4 weights and change only the KV recipe.

NEW COMPACT PREFILL RESULTS / OCTOBER 16, 2026

KV savings persist with INT4 weights. Speed depends on batch size.



Qwen3-8B-AWQ: same INT4 weights, FP16 keys + FP8 values.

Batch size (requests)	Request throughput vs FP16 KV baseline
1	+0.65%
8	+2.28%
32	−1.44%

−25%

persistent KV payload

58 vs 44

resident requests

Separate capacity test; workspace paid.

AWQ quality checks pass.

PR QUEUE

Asymmetric Quantization upstream roadmap

vLLM Independent K/V dtypes, allocation and dispatch - Goes last

FlashInfer Attention kernels that consume the representation - PR submitted

LMCache Preserve K/V planes, scales and layout on save / restore #3710



Experimental branches. Representation and lifecycle must agree end to end.

LMCache DMA-BUF + io_uring

The clean vendor neutral Linux p2p path GPU ← → NVMe

Today's P2P path:

- NVIDIA GDS:
  - Legacy: Proprietary nvidia GPU driver + cuFile + nvida-fs + custom NVMe: does not work on newer kernels
  - Current: OpenRM nvidia GPU driver (not upstream) + cuFile + P2PDMA
- AMD AIS:
  - hipFile + P2PDMA + Linux AMD Infinity Storage (AIS) - not upstream

Tomorrow:

- NVIDIA:
  - Nouveau: export works, requires testing
  - Nova: requires testing and dma-buf export support and validation
  - NVIDIA OpenRM GPU driver: cross OS driver CUDA solution – current main test route
    - CUDA dma-buf export works:
      - CUDA + io-uring dma-buf
    - CUDA dma-buf export does not work:
      - rust_raw_block.gpu_buffer_provider: vulkan_rm
      - CUDA + vulcan for alloc | hipFile + io-uring dma-buf
- AMD:
  - ROCm dma-buf export + io-uring dma-bu
- Why not Vulcan + io-uring dma-buf for all

The P2P PCI support is optional

- PCIe 4.0 §1.3.1 - optional support for P2P
  - Linux white list
    - Each root complex needs validation
- Expect bug reports about this
  - Undefined behaviour if you try to add your root complex to the whitelist
- I expect inference pressure to streamline this support
  - High end servers
  - Consumer GPUs
  - Laptops
  - Mobile

The P2P PCI support is optional

- PCIe 4.0 §1.3.1 - optional support for P2P
  - Linux white list
    - Each root complex needs validation
- Expect bug reports about this
  - Undefined behaviour if you try to add your root complex to the whitelist
- I expect inference pressure to streamline this support
  - High end servers
  - Consumer GPUs
  - Laptops
  - Mobile
- How about PCI multicast use cases?

Can LMCache support PD disaggregation directly

Today LMCache supports PD disaggregation via NIXL

- NIXL connector

The chain of events:

- Add NIXL io-uring dma-buf support

Can LMCache support PD disaggregation directly

Today LMCache supports PD disaggregation via NIXL

- NIXL connector

The chain of events:

- Add NIXL io-uring dma-buf support
- Why go through NIXL?

Can LMCache support PD disaggregation directly

Today LMCache supports PD disaggregation via NIXL

- NIXL connector

The chain of events:

- Add NIXL io-uring dma-buf support
- Why go through NIXL?

Can we support PD disaggregation directly on LMCache?

- Yes

Can LMCache support PD disaggregation directly

Today LMCache supports PD disaggregation via NIXL

- NIXL connector

The chain of events:

- Add NIXL io-uring dma-buf support
- Why go through NIXL?

Can we support PD disaggregation directly on LMCache?

- Yes

Should LMCache support it?



PUBLIC RFC / BOUNDED VALIDATION

DMA-BUF io-uring + PD LMCache RFC posted

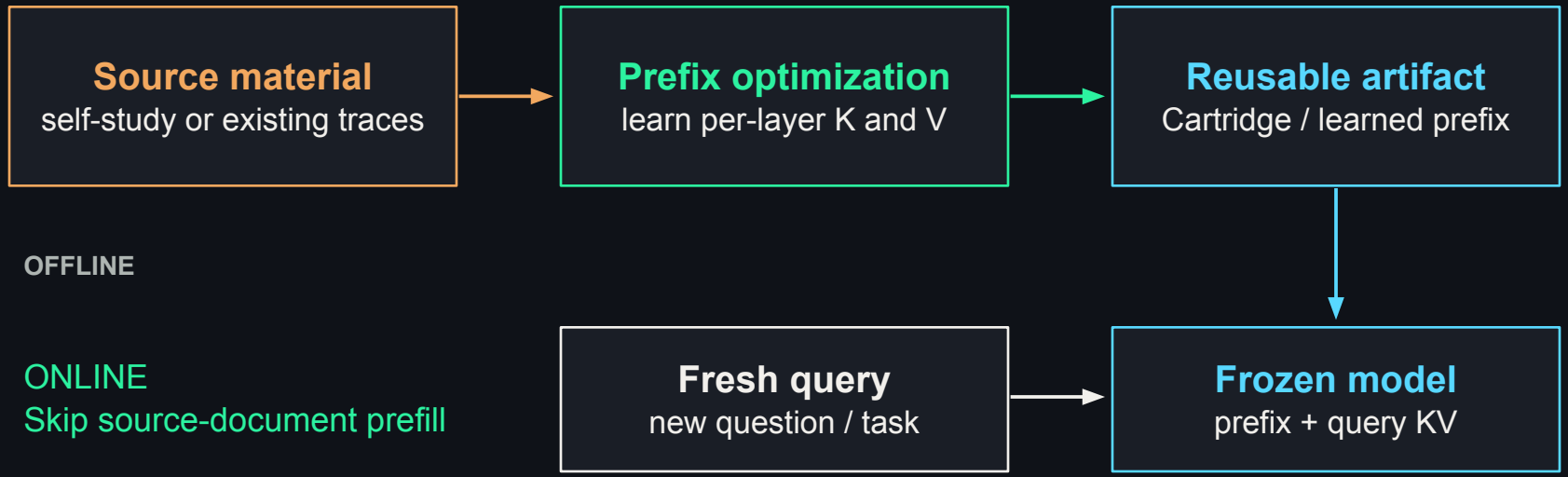
- Go help review it
- We need a lot of testing

RFC #5816



Prefix tuning: a prefix cache can be learned

Documents (Cartridges), reasoning traces (ReasonCACHE for CoT) provide different training inputs.
Cartridges At Scale (CAS): Amazon scaled it for RAG use cases and blending



The model stays frozen. The new query still needs prefill.

A model may want to offload more

- Cartridges: compressed KV cache → Prefix Tuning
- Reason Cache: CoT → Prefix Tuning

What is the difference? How they are created

Is that it?

A model may want to offload more

- Cartridges: compressed KV cache → Prefix Tuning
- Reason Cache: CoT → Prefix Tuning

What is the difference? How they are created

Is that it?

- REFRAG: MLP learns to interpret a small embedding for a document

A model may want to offload more

- Cartridges: compressed KV cache → Prefix Tuning
- Reason Cache: CoT → Prefix Tuning

What is the difference? How they are created

Is that it?

- REFRAG: MLP learns to interpret a small embedding for a document
- Memorilla: Frozen model, model query learns to get relevant embeddings

A model may want to offload more

- Cartridges: compressed KV cache → Prefix Tuning
- Reason Cache: CoT → Prefix Tuning

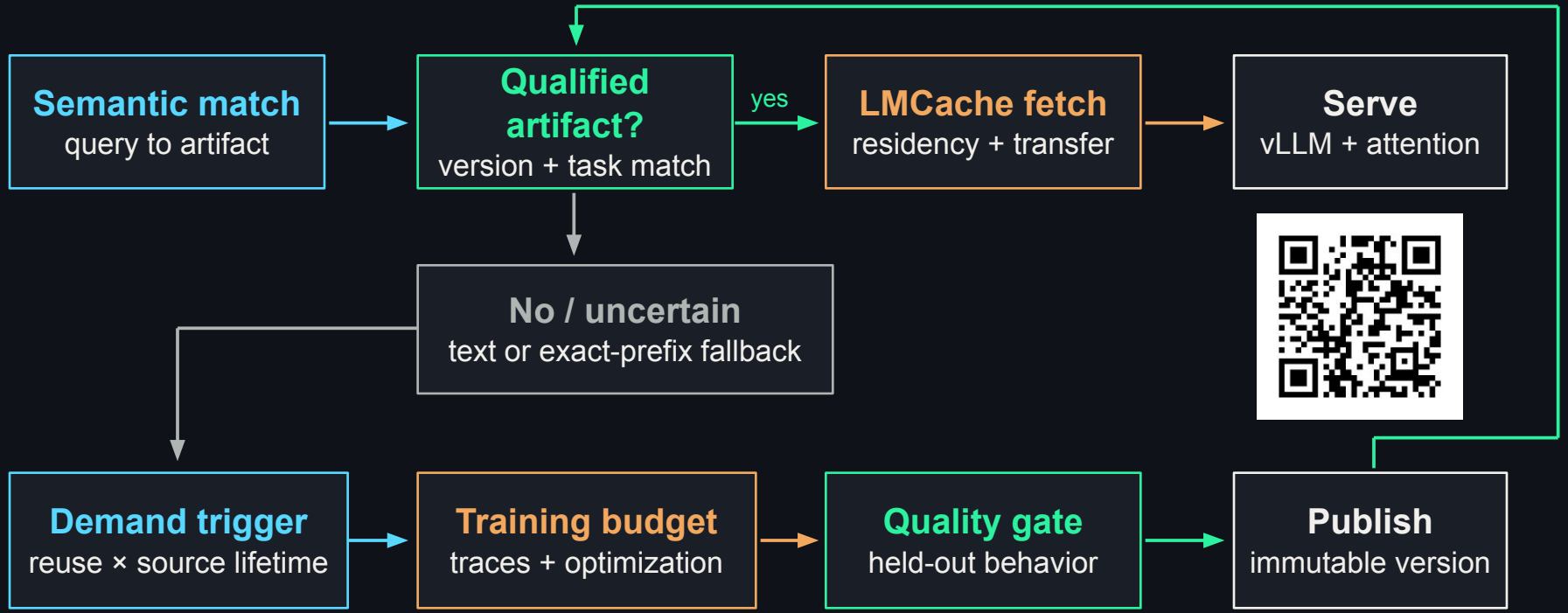
What is the difference? How they are created

Is that it?

- REFRAG: MLP learns to interpret a small embedding for a document
- Memorilla: Frozen model, query learns to get relevant embeddings
  - Both can be cached, but have custom bespoke solutions

A PROPOSED SYSTEM DECOMPOSITION

Association, economics, quality and plumbing

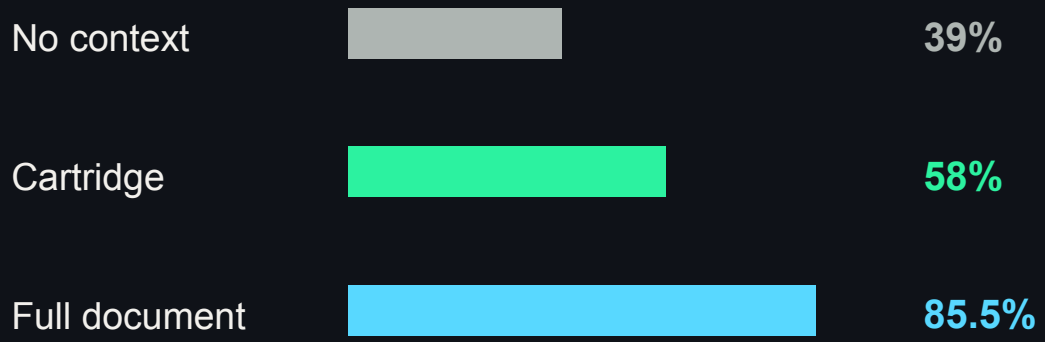


Router: meaning and demand. LMCache: residency and bytes. Engine: execution.

QUALITY AND ECONOMICS

Training cost and quality remain the gates

Answer accuracy, Qwen3-8B / LongHealth
Five isolated cartridges, 2,048-token cap



Independent CAS reproduction. A quality gap remains.

8.21–8.29

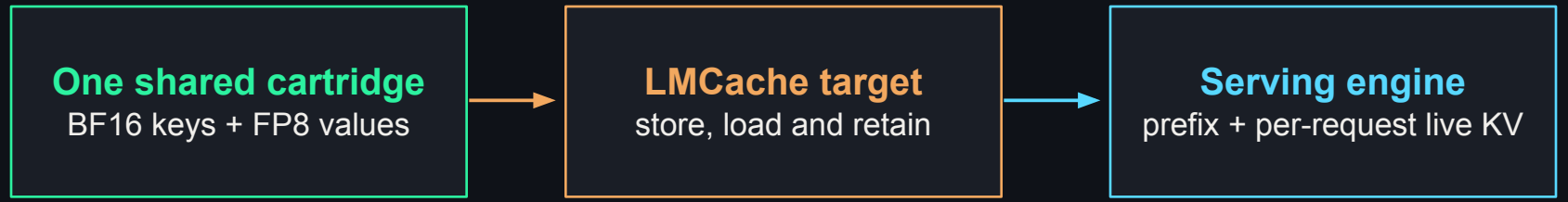
H100-hours
per measured training job

One patient_02 artifact.
Self-study cost excluded.
Retries, updates and cold loads add cost.

Compare against a strong text + exact-prefix-cache baseline. Break-even is unproven.

THE THREE TOPICS MEET

Cartridges connect all three topics



25% fewer

raw cartridge bytes
88.875 MiB → 66.656 MiB example

Paired in-sample loss change
K16/V8: +0.029–0.038%
K8/V8: +0.873–1.303%
Three trained artifacts. This is not task accuracy.

Cartridge quality needs separate qualification from ordinary runtime KV.

NEARER RESEARCH

Sharing state across related models

The opportunity. There is R&D on sharing a KV Cache. It may transfer to Cartridges.

Evaluations. DroidSpeak, KV-Lingo, Mixture of Translators

Public cache. Would a Cartridges CDN for public popular RAG documents be useful? Quality and reproduction artifacts needs to be addressed. How about exploits?

Learned-cartridge transfer has not been demonstrated.

NEARER RESEARCH

Sharing state across related models



The opportunity. There is R&D on sharing a KV Cache. It may transfer to Cartridges. Not yet.

Evaluations. DroidSpeak, KV-Lingo, Mixture of Translators

Public cache. Would a Cartridges CDN for public popular RAG documents be useful? Quality and reproduction artifacts needs to be addressed. How about exploits?

Could a model learn to generate a compressed cache?
Does Memorilla address this need?

Learned-cartridge transfer has not been demonstrated.

DISCUSSION

Questions?